Docker. Una nueva forma de ejecutar y desarrollar aplicaciones

Cloud-DI Team - Departamento de Informática. UAL

Tabla de contenidos

Resumen . . . 2
1. Conceptos básicos . . . 3
  1.1. El tópico . . . 3
  1.2. Conviene no confundir . . . 3
  1.3. Qué es Docker . . . 5
  1.4. Docker vs Máquinas virtuales . . . 6
  1.5. Ventajas . . . 7
  1.6. Contenedores e imágenes . . . 7
2. Un ejemplo sencillo . . . 8
  2.1. Antes de nada . . . 8
  2.2. Docker engine . . . 8
  2.3. El Hola mundo . . . 9
  2.4. Crear un contenedor Apache . . . 9
  2.5. Funcionamiento básico con Docker . . . 9
  2.6. Imágenes interesantes de Docker . . . 10
  2.7. Operaciones sobre contenedores . . . 10
  2.8. Resumen de comandos básicos para contenedores . . . 13
3. Creación de imágenes propias . . . 15
  3.1. El `Dockerfile` . . . 15
  3.2. Imágenes . . . 16
  3.3. Ejemplo de contenedor para aplicaciones web en PHP . . . 16
  3.4. Ejemplo de contenedor con volumen externo . . . 17
  3.5. Descarga y subida de imágenes a Docker Hub . . . 18
  3.6. Resumen de comandos básicos para imágenes . . . 18
4. Aplicaciones con varios contenedores . . . 20
  4.1. Flujo de trabajo básico con Docker Compose . . . 20
  4.2. Comandos básicos para Docker Compose . . . 20
5. Ejemplo: Aplicación web (PHP) con soporte de Base de datos (MySQL) . . . 21
6. Plataformas de gestión de contenedores . . . 23
  6.1. Administración de Docker con *Portainer* . . . 23
  6.2. Administración de Docker con Rancher 1.6 (Rancher Server) . . . 24
7. Escalado de aplicaciones con Docker Swarm . . . 26
  7.1. Orquestación de contenedores . . . 26
  7.2. Orquestadores de contenedores más populares . . . 26
  7.3. Docker Swarm . . . 26
  7.4. Recuperación automática ante errores (*Self-healing*) . . . 31
  7.5. Escalado de la aplicación . . . 32
  7.6. Resumen de los comandos para *swarms* . . . 34

8. Docker machine . . . 35
8.1. Creación de archivo con variables de entorno . . . 35
8.2. Creación de los nodos del *swarm* . . . 35
8.3. Comprobación del *swarm* creado . . . 37
8.4. Despliegue de la aplicación en los nodos . . . 37
8.5. Desalojo de un nodo . . . 40
9. Creación de clusters Kubernetes con Rancher 2.0 . . . 42
10. Otras cosas interesantes . . . 44
10.1. Microservicios y contenedores . . . 44
10.2. El ecosistema Docker . . . 45
10.3. Instalar un registro de imágenes propio . . . 47
10.4. Conexión a daemons Docker remotos . . . 48
10.5. Uso de HAProxy . . . 48

DEPARTAMENTO
DE INFORMÁTICA
UNIVERSIDAD DE ALMERÍA

Resumen

Docker es un proyecto open source creado en 2013 y que ha supuesto una revolución para el desarrollo y despliegue de operaciones. Docker abstrae el hardware y el sistema operativo del host ejecutando las aplicaciones en contenedores, compartimentos aislados que contienen todos los recursos para una aplicación o servicio.

En este seminario veremos cómo usar Docker para el desarrollo de aplicaciones sencillas, aprendiendo a crear servicios con *Docker Compose*, clusters con *Docker Swarm* e interactuar de forma remota con *Docker machine*

Objetivos

- Conocer los componentes básicos de Docker
- Crear contenedores a partir de imágenes de Docker Hub
- Aprender a usar `Dockerfile` para la creación de imágenes
- Usar *Docker Compose* para construir entornos de contenedores
- Usar volúmenes para almacenamiento persistente
- Estudiar *Docker Swarm* para el escalado de aplicaciones
- Crear clusters con *Docker Machine*
- Estudiar ejemplos de entornos elásticos

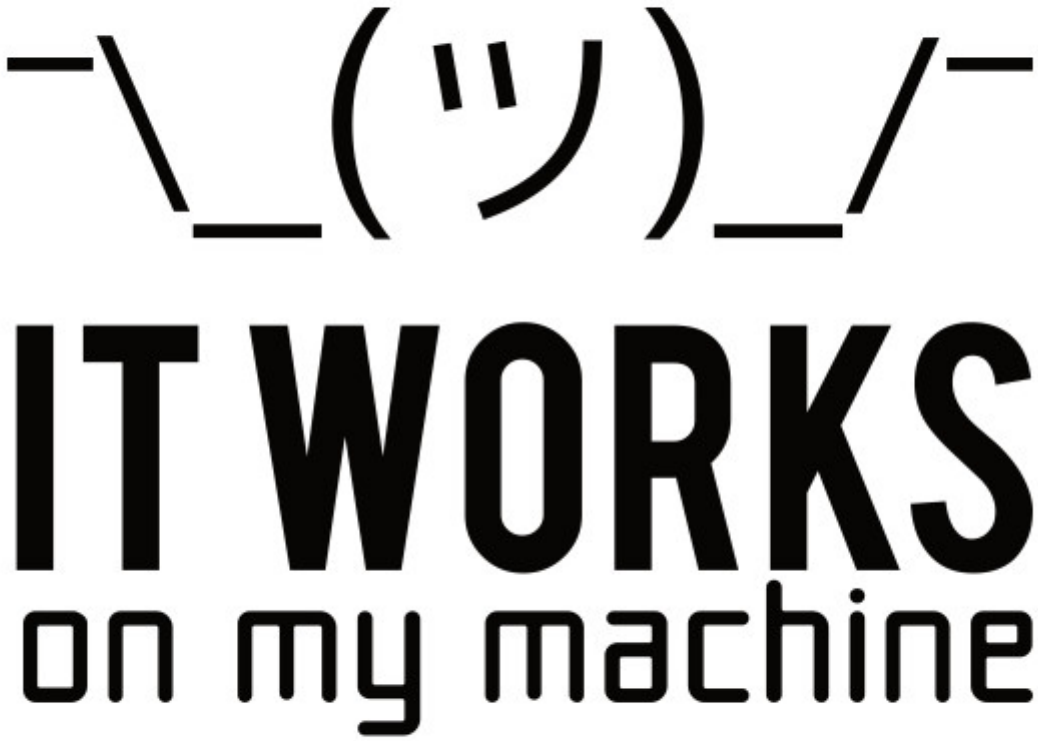
TIP

Disponibles los repositorios usados en este seminario:

- Catálogo de clientes: https://github.com/ualmtorres/docker_customer_catalog
- Swarm: https://github.com/ualmtorres/swarm

Chapter 1. Conceptos básicos

1.1. El tópico



1.2. Conviene no confundir

Figure 1. Contenedores El Morales es una empresa almeriense de alquiler de contenedores de obra

Figure 2. Dockers es una marca de prendas de ropa y calzado

Figure 3. El Vallenato es un género musical colombiano

1.3. Qué es Docker

- Docker es una plataforma para que desarrolladores y administradores puedan desarrollar, desplegar y ejecutar aplicaciones en un entorno aislado denominado contenedor.
- Docker permite separar las aplicaciones de la infraestructura acelerando el proceso de entrega de software a producción.
- Proyecto open source creado en 2013 que hace uso de LXC (Linux Containers). LXC es un método de virtualización a nivel de S.O.

TIP	Docker permite empaquetar una aplicación con todas sus dependencias para que pueda ser ejecutada en plataformas diferentes. **El proceso de despliegue es rápido y repetible.**

Basta con ejecutar los tres comandos siguientes en una máquina con Docker instalado para tener una aplicación web que muestra un catálogo de clientes almacenados en una base de datos MySQL.

```
$ git clone https://github.com/ualmtorres/docker_customer_catalog.git
$ cd docker_customer_catalog
$ docker-compose up -d
```

Web PHP-MySQL con Docker

localhost

Docker app

Ejemplo de aplicacion PHP y MySQL con contenedores

Usa un contenedor para Apache/PHP y otro para MySQL con almacenamiento de aplicación y de datos en volúmenes externos

Name	Credit Rating	Address	City	State	Country	Zip
One Sport	EXCELLENT	82 Via Bahia	Sao Paolo		Brazil	
Deportivo Caracas	EXCELLENT	31 Sabana Grande	Caracas		Venezuela	
New Delhi Sports	GOOD	11368 Chanakya	New Delhi		India	
Ladysport	EXCELLENT	281 Queen Street	Seattle	Washington	US	
Kim's Sporting Goods	EXCELLENT	15 Henessey Road	Hong Kong			

Figure 4. Aplicación sencilla que muestra un listado de clientes de una base de datos

1.4. Docker vs Máquinas virtuales

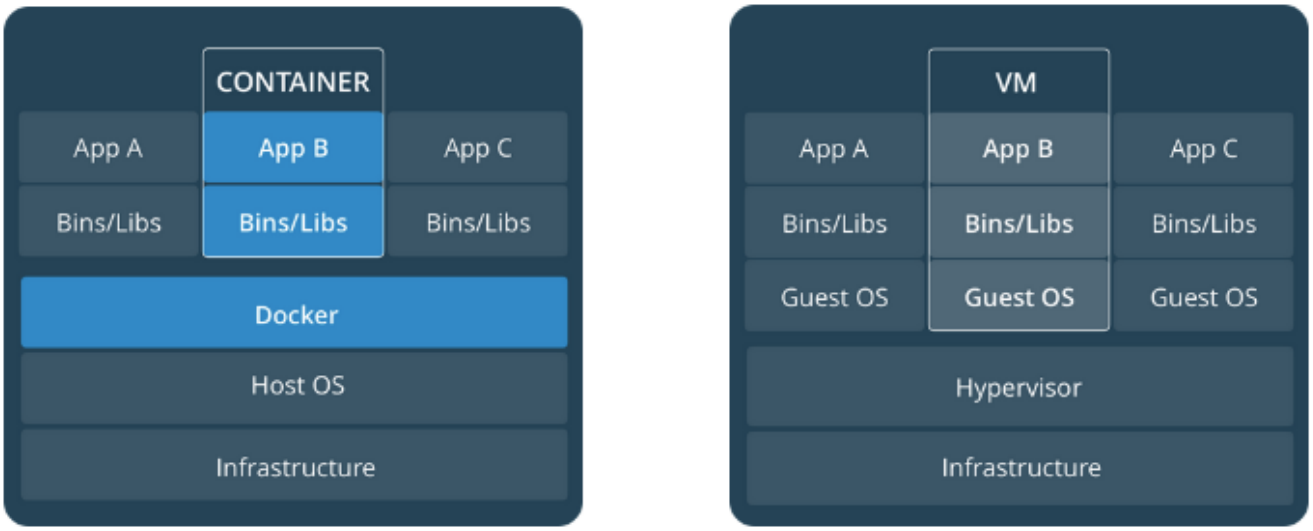


Figure 5. Docker vs Máquinas virtuales

- Una máquina virtual proporciona un entorno con más recursos de los que necesitan la mayoría de las aplicaciones

- Mayor número de contenedores que de MV en el mismo hardware.
- Los contenedores se pueden ejecutar en hosts que sean máquinas virtuales.

1.5. Ventajas

- Ligeros: Los contenedores comparten el kernel del host.
- Intercambiables: Despliegue de actualizaciones en caliente.
- Portables: Build local y ejecución en cualquier lugar.
- Escalables: Aumento y distribución automática de réplicas de contenedores.
- Apilables: Aumento del stack de servicios en caliente.

NOTE	Docker supone una revolución en los entornos de CI/CD. Tras la actualización del repositorio de proyecto, se crean contenedores para pasar las pruebas, se construyen las nuevas imágenes y se despliega la nueva versión de la aplicación **sin parada del sistema**.

1.6. Contenedores e imágenes

- Un contenedor se lanza ejecutando una imagen.
- Una imagen es una plantilla con las instrucciones de creación de un contenedor Docker:
  - Código
  - Runtime
  - Librerías
  - Variables de entorno
  - Archivos de configuración

Chapter 2. Un ejemplo sencillo

2.1. Antes de nada

2.1.1. Instalación:

https://docs.docker.com/install/#desktop

Obtenemos:

- Daemon de docker
- Cliente de docker
- Docker compose

2.1.2. Crear cuenta en Docker Hub

Docker Hub es un registro público de imágenes (Lugar donde se almacenan imágenes): https://hub.docker.com

NOTE	Docker Hub permite en su plan libre tener un repositorio privado de imágenes. También permite automatizar la construcción de imágenes y su despliegue con repositorios GitHub y Bitbucket

2.2. Docker engine

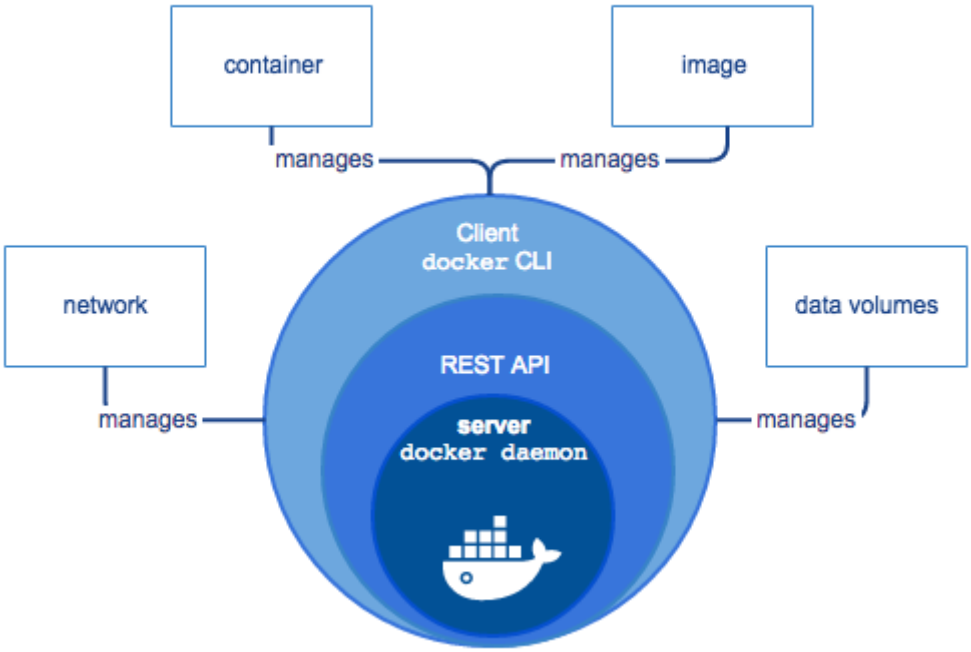


Figure 6. Componentes de Docker Engine

2.3. El Hola mundo

```
$ docker --version
Docker version 18.03.1-ce, build 9ee9f40

$ docker run hello-world
Unable to find image 'hello-world:latest' locally
latest: Pulling from library/hello-world
9bb5a5d4561a: Pull complete
Digest: sha256:f5233545e43561214ca4891fd1157e1c3c563316ed8e237750d59bde73361e77
Status: Downloaded newer image for hello-world:latest

Hello from Docker!
This message shows that your installation appears to be working correctly.
....
```

2.4. Crear un contenedor Apache

```
$ docker run -d -p 82:80 httpd
```

- Descarga una imagen si no existe localmente, lanza un contenedor y asocia el puerto 82 del host al puerto 80 del contenedor
- `-d` Lanza el contenedor en modo *detached*

NOTE	El primer puerto que aparece es el del host y el segundo el del contenedor
TIP	También podemos usar el parámetro `--name <nombre>` para darle un nombre al contenedor. De forma predeterminada, Docker asigna un nombre aleatorio a los contenedores creados. El asignar un nombre es útil para administrar posteriormente los contenedores (pausa, eliminación, ...)

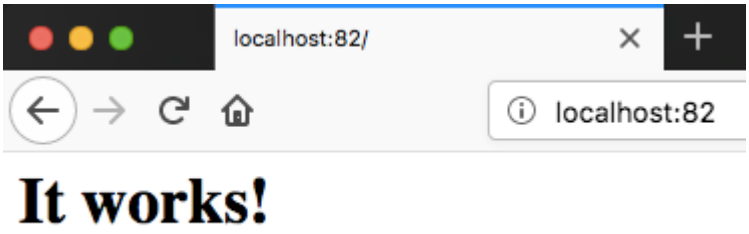


Figure 7. Contenedor ejecutando Apache

2.5. Funcionamiento básico con Docker

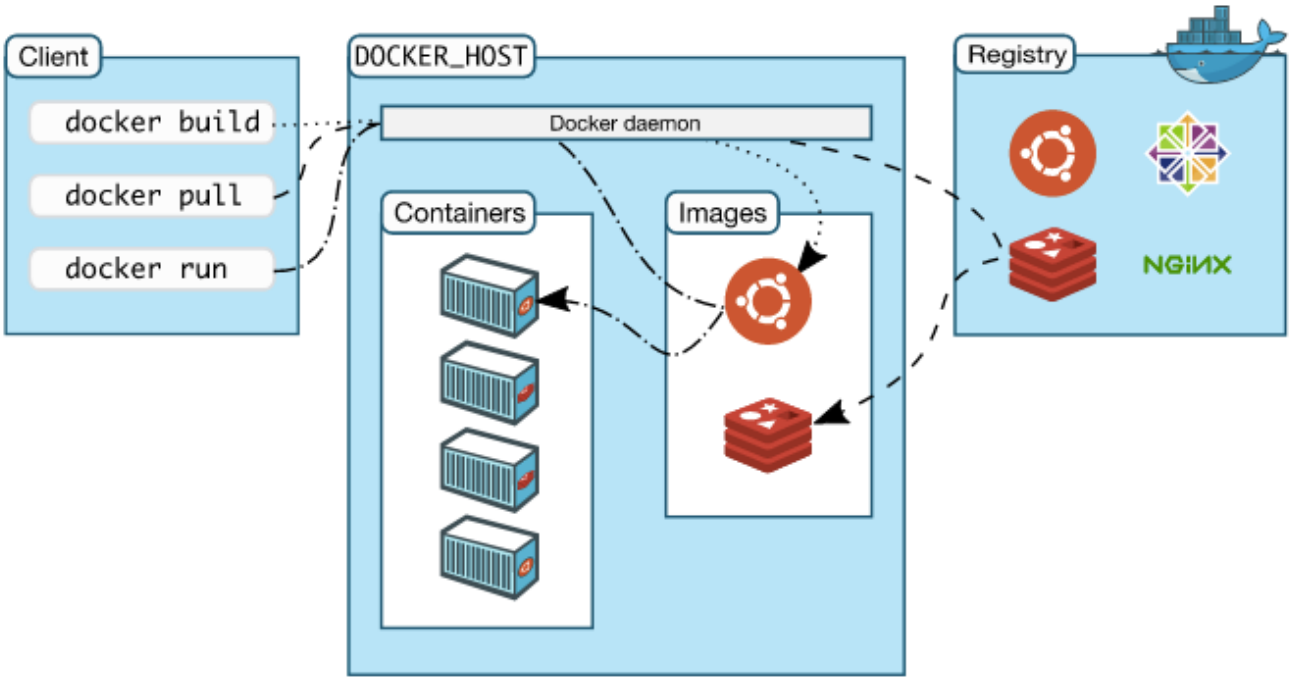


Figure 8. Funcionamiento básico con Docker

2.6. Imágenes interesantes de Docker

En https://hub.docker.com/explore/ se encuentran las imágenes ordenadas por popularidad. Destacamos:

- alpine: Linux reducido
- nginx: Servidor web Nginx
- httpd: Servidor web Apache
- ubuntu: Ubuntu
- redis: Base de datos Redis (clave-valor)
- mongo: Base de datos MongoDB (documentos)
- mysql: Base de datos MySQL (relacional)
- postgres: Base de datos PostgreSQL (relational)
- node: Node.js
- registry: Registro de imágenes on-premise
- php, elasticsearch, haproxy, WordPress, rabbitmq, python, openjdk, tomcat, jenkins, redmine, flink, spark, ...

2.7. Operaciones sobre contenedores

2.7.1. Mostrar contenedores

```
$ docker ps
CONTAINER ID        IMAGE               COMMAND                  CREATED
```

```
STATUS              PORTS                    NAMES
d2f73e6acd51        httpd                "httpd-foreground"       11 minutes ago
Up 11 minutes       0.0.0.0:82->80/tcp       upbeat_stonebraker
```

NOTE Los nombres generados para los contenedores son aleatorios si no se usa el parámetro `-name` al crearlos.

2.7.2. Detener y reanudar contenedores

Primero, obtener con `docker ps` el `CONTAINER ID` del contenedor que queremos detener.

```
$ docker ps
CONTAINER ID        IMAGE               COMMAND                  CREATED
STATUS              PORTS                    NAMES
d2f73e6acd51        httpd               "httpd-foreground"       11 minutes ago
Up 11 minutes       0.0.0.0:82->80/tcp       upbeat_stonebraker
```

Detener el contenedor

```
$ docker stop d2f73e6acd51
```

CAUTION Al hacer `docker ps` no se muestran los contenedores que estén detenidos.

Mostrar todos los contenedores, también los detenidos

```
$ docker ps -a
CONTAINER ID        IMAGE               COMMAND                  CREATED
STATUS                        PORTS                    NAMES
d2f73e6acd51        httpd               "httpd-foreground"       20 minutes ago
Exited (0) 2 minutes ago                               upbeat_stonebraker
```

Reanudar un contenedor

```
$ docker start d2f73e6acd51
```

Tras reanudar el contenedor, vuelve a aparecer cuando hacemos `docker ps`

```
$ docker ps
CONTAINER ID        IMAGE               COMMAND                  CREATED
STATUS              PORTS                    NAMES
d2f73e6acd51        httpd               "httpd-foreground"       9 hours ago
Up 10 seconds       0.0.0.0:82->80/tcp       upbeat_stonebraker
```

Detener todos los contenedores en ejecución

Primero obtenemos los identificadores de los contenedores en ejecución con `docker ps -q`. Ese comando lo podemos encerrar entre apóstrofes y pasar su resultado a otro comando en la misma línea.

```
$ docker stop `docker ps -q`
```

Iniciar una lista de contenedores

```
$ docker start d2f73e6acd51 9811efbf6e45 178c2d03f2e7
```

2.7.3. Abrir un terminal en un contenedor

```
$ docker exec -it d2f73e6acd51 bash
root@d2f73e6acd51:/usr/local/apache2#
```

Se inicia una sesión como `root` en el contenedor. En la terminal del contenedor podemos ejecutar comandos del sistema operativo (`ls, df -h, cat /proc/cpuinfo, ⋯`). La cantidad y el tipo de comandos dependerá de la imagen usada para crear el contenedor.

2.7.4. Copia de datos

CAUTION	El almacenamiento en un contenedor no es persistente. Se eliminan los datos escritos en él tras su eliminación.

```
docker cp [OPTIONS] CONTAINER:SRC_PATH DEST_PATH|-
docker cp [OPTIONS] SRC_PATH|- CONTAINER:DEST_PATH
```

Como ejemplo vamos a crear en nuestro host un archivo `index.html` y lo copiaremos en el contenedor para sustituir la página de inicio del servidor Apache.

```
<!-- Ejemplo de archivo index.html -->
<html lang="es">
  <body>
    <h1>Docker es una maravilla</h1>
  </body>
</html>
```

Ahora copiamos el archivo `index.html` al contenedor con `docker cp`

```
$ docker cp index.html d2f73e6acd51:/usr/local/apache2/htdocs/
```

Figure 9. Cambio de página de inicio

2.7.5. Eliminación de un contenedor

Primero paramos el contenedor con `docker stop` y luego lo eliminamos con `docker rm`

```
$ docker stop d2f73e6acd51
$ docker rm d2f73e6acd51
```

También se puede eliminar directamente un contenedor en ejecución forzando su eliminación

`$ docker rm -f <id>`

Al crear un nuevo contenedor a partir de la imagen `httpd` comprobamos que la página de inicio modificada anteriormente se eliminó junto al contenedor eliminado.

```
$ docker run -d -p 82:80 httpd
```

TIP

Podemos eliminar todos los contenedores creados a partir de una imagen con la secuencia de comandos siguiente (p. ej. eliminar todos los contenedores creados a partir de una imagen `wordpress`)

```
$ docker rm -f `docker ps -a | grep "wordpress" | awk '{print $1}'`
```

2.8. Resumen de comandos básicos para contenedores

```
$ docker info
$ docker version
$ docker run <image> // Crea un contenedor a partir de una imagen. Si no tenemos la
imagen en local, la descarga
$ docker run -d -p 82:80 nginx: Crea un contenedor en modo deattached accesible desde
el puerto 82
$ docker stop|start <id>: Detiene|Continúa un contenedor
$ docker ps -a: Listado de contenedores (-a muestra también los parados)
$ docker ps -q: Listado de los ids de los contenedores
$ docker stop `docker ps -q`: Para todos los contenedores que devuelve el subcomando
`docker ps -q`
$ docker rm <id>: Borra un contenedor si está parado
```

```
$ docker rm -f <id>: Fuerza el borrado de un contenedor aunque esté parado
$ docker exec -it <id> sh: Abre una terminal en el contenedor
$ docker exec <id> ls: Ejecuta el comando ls en el contenedor para mostrar sus
archivos
$ docker cp <id>:./dockerenv .: Copia el fichero dockerenv del contenedor en nuestro
sistema de archivos local
$ docker rm -f `docker ps -a | grep "wordpress" | awk '{print $1}'`: Eliminar todos
los contenedores creados a partir de una imagen
```

Chapter 3. Creación de imágenes propias

3.1. El `Dockerfile`

- Para construir una imagen, se crea un `Dockerfile` con las instrucciones que especifican lo que va a ir en el entorno, dentro del contenedor (redes, volúmenes, puertos al exterior, archivos que se incluyen.
- Indica cómo y con qué construir la imagen.
- Conseguimos que el build de la aplicación definida en el contenedor se comporte de la misma forma en cualquier lugar que se ejecute. Hacemos que sea repetible.

Ejemplo de `Dockerfile`

```
# Use an official Python runtime as a parent image
FROM python:2.7-slim

# Set the working directory to /app
WORKDIR /app

# Copy the current directory contents into the container at /app
ADD . /app

# Install any needed packages specified in requirements.txt
RUN pip install --trusted-host pypi.python.org -r requirements.txt

# Make port 80 available to the world outside this container
EXPOSE 80

# Define environment variable
ENV NAME World

# Run app.py when the container launches
CMD ["python", "app.py"]
```

Fragmento de `Dockerfile` para construir una imagen con Ubuntu como base y definiendo dónde se montará un volumen externo

```
FROM ubuntu:latest
RUN apt-get update -y
RUN apt-get install -y python-pip python-dev
WORKDIR /app
ENV DEBUG=True
EXPOSE 80
VOLUME /data ①
```

① Crea un punto de montaje en el contenedor. A la hora de crearlo le haremos corresponder

normalmente un directorio del host

3.2. Imágenes

- Se construyen con `docker build` a partir de un `Dockerfile`
- Se crean en un contexto (normalmente añadiendo archivos del directorio de trabajo del host a la imagen -p. ej. el código fuente de la aplicación)
- Con `FROM` (normalmente primera instrucción del `Dockerfile`) inicializamos el sistema de archivos de la imagen (p.e. si es ubuntu obtenemos el sistema de archivos de Ubuntu)
- Muchas imágenes disponibles en Docker Hub usan Alpine (una distribución ligera de Linux) en lugar de Ubuntu, Fedora o CentOS, debido a su menor tamaño
- Cada instrucción del `Dockerfile` genera una nueva capa (con la diferencia) en ese sistema de archivos
- Al hacer `build` las capas existentes en el registro local no se vuelven a crear

NOTE Una imagen comprimida de Alpine está en torno a los 2 MB, mientras que una imagen comprimida de Ubuntu está entre 40 y 80 MB

3.3. Ejemplo de contenedor para aplicaciones web en PHP

Vamos a construir un contenedor que incluya de forma estática una aplicación (p. ej. la última versión de la aplicación). El proceso a seguir es:

1. Creación de la aplicación.
2. Creación del `Dockerfile` para generación de la imagen.
3. Generación de la imagen.

A partir de una carpeta nueva crearemos lo siguiente:

- Archivo `Dockerfile`
- Carpeta `html` con los scripts de nuestra aplicación
- Archivo `html/index.php` con el código de nuestra aplicación

El `Dockerfile`

```
FROM ualmtorres/phalcon-apache-ubuntu

ADD html /var/www/html

EXPOSE 80
```

Archivo `html/index.php` de ejemplo

```
<?php
  echo "Hola desde Docker";
?>
```

3.3.1. Construcción de la imagen.

`$ docker build -t pruebaphp .`

Con `-t` definimos una etiqueta o nombre de la imagen. Al construir la imagen pasa a nuestro registro local.

3.3.2. Listado de imágenes locales

```
$ docker image ls
REPOSITORY            TAG                 IMAGE ID            CREATED
SIZE
pruebaphp             latest              152781e32617        14 hours ago
245MB
```

3.3.3. Creación de un contenedor a partir de la imagen

`$ docker run -d -p 83:80 pruebaphp`

Un posible inconveniente que podemos encontrar en este ejemplo es que la aplicación va incluida en la propia imagen, por lo que para actualizar la aplicación deberemos crear una nueva imagen, y después crear un nuevo contenedor a partir de ella desechando el contenedor anterior.

NOTE	A la hora de distribuir y actualizar aplicaciones podemos incluir la aplicación en la imagen. Con un ciclo de CI/CD tendríamos la aplicación actualizada al actualizar su repositorio.

3.4. Ejemplo de contenedor con volumen externo

En este ejemplo la aplicación la tendremos aparte en un volumen externo accesible por el contenedor. De esta forma, si nuestra aplicación está vinculada a un repositorio, la actualización de la aplicación se realiza descargando la última versión del repositorio, manteniendo intacto el contenedor.

La forma de usar volúmenes con `Dockerfile` consiste en:

1. Añadir en el `Dockerfile` la lista de carpetas que se montarán con volúmenes externos
2. Al crear el contenedor indicar el punto de montaje en el host remoto en forma de ruta absoluta

El `Dockerfile`

```
FROM ualmtorres/phalcon-apache-ubuntu
```

```
VOLUME /var/www/html

EXPOSE 80
```

```
$ docker run -d -p 83:80
-v=/Users/manolo/Documents/Desarrollo/SeminarioDocker/phpsimple/html:/var/www/html
pruebaphp
```

TIP

También podemos hacer uso de la evaluación de órdenes con apóstrofes para obtener el path actual y añadirle solo la carpeta `html`.

```
$ docker run -d -p 83:80 -v=`pwd`/html:/var/www/html pruebaphp
```

3.5. Descarga y subida de imágenes a Docker Hub

- Etiquetar la imagen antes de subirla a Docker Hub

```
$ docker tag phpprueba ualmtorres/phpprueba:v0
```

- Subida de la imagen a Docker Hub

```
docker push <usuario>/<image>
```

- Al hacer `push` las capas que ya estén subidas no se vuelven a subir. Cuando una instrucción del `Dockerfile` cambia una capa, invalida al resto y hay que volver a generar las instrucciones de las capas restantes. Por tanto, colocaremos antes en el `Dockerfile` lo que menos cambie.
- Al hacer `pull` sólo se descargan las capas nuevas.
- Si cambiamos en el host archivos de los que se incluyen en la imagen se genera una capa nueva invalidando la caché.

```
$ docker pull wordpress
$ docker run -d -p 80:80 --name my_wordpress wordpress
```

3.6. Resumen de comandos básicos para imágenes

```
$ docker login
$ docker run -d ngninx
$ docker pull <image>
$ docker image ls: Lista imágenes locales
```

```
$ docker inspect <image>: Propiedades de una imagen
$ docker image rm <image>: Elimina una imagen local
```

Chapter 4. Aplicaciones con varios contenedores

- Docker Compose es una herramienta para definir y ejecutar aplicaciones Docker con varios contenedores.
- Usaremos un archivo `docker-compose.yml` para configurar los *servicios* de la aplicación. Los servicios son las partes de la aplicación (p. ej. un servicio para el almacenamiento de los datos y otro para el front-end)
- En un mismo host podemos tener varios entonos aislados. Compose usa nombres de proyecto para mantener a los entornos aislados. De forma predeterminada se usa el nombre del directorio desde donde se lanza la aplicación.
- `docker-compose --version` Para obtener la versión y saber si está instalado.
- Instalación desde https://docs.docker.com/compose/install

4.1. Flujo de trabajo básico con Docker Compose

1. Crear el archivo `docker-compose.yml` con los servicios de la aplicación (p. ej. php y mysql)
2. Construir y lanzar el entorno en modo *detached* con `docker-compose up -d`
3. Echar abajo el entorno con `docker-compose down`

4.2. Comandos básicos para Docker Compose

```
$ docker-compose up -d      Construye y lanza el entorno en modo dettached
$ docker-compose pull       Descarga las imágenes pero no inicia los contenedores
$ docker-compose rm [-fs]   Borra los contedores parados. Con -fs los detiene y fuerza
su borrado
```

Chapter 5. Ejemplo: Aplicación web (PHP) con soporte de Base de datos (MySQL)

- Aplicación que muestra un listado de clientes almacenado en una base de datos MySQL.
- Podemos distribuirla con un repositorio que incluya una carpeta `html` con la aplicación PHP.
- Al iniciar el servicio MySQL se ejecutará un script de inicialización de la base de datos.
- Usaremos volúmenes externos para la base de datos y para la aplicación web para asegurar la persistencia de los cambios.

Comencemos clonando el repositorio de la aplicación:

```
$ git clone https://github.com/ualmtorres/customer_catalog.git
```

En ese repositorio se encuentra:

- Un archivo `docker-compose.yml` que configura dos servicios. Un servicio para almacenamiento de datos con MySQL y otro servicio para el front-end PHP de la aplicación.
- Una carpeta `html` con la aplicación. Esta carpeta será la que monte el servicio front-end, de forma que la aplicación no está almacenada en el contenedor.
- Un script SQL `init.sql` que inicializa la base de datos de nuestra aplicación. La base de datos se almacena en nuestro host, garantizando almacenamiento persistente.

`docker-compose.yml`

```
version: '2'
services:
  mysql:
    container_name: my_mysql
    restart: always
    image: mysql:5.7
    environment:
      MYSQL_ROOT_PASSWORD: 'secret' # TODO: Change this
    ports:
      - "3306:3306"
    volumes:
      - ./data:/var/lib/mysql ①
      - ./init.sql:/docker-entrypoint-initdb.d/init.sql ②
  php:
    container_name: my_php
    restart: always
    image: ualmtorres/phalcon-apache-ubuntu
    ports:
      - "80:80"
    volumes:
      - ./html:/var/www/html ③
```

① Montar una carpeta `data` de nuestro host en la ruta en la que el servicio `mysql` almacena la base de datos

② La imagen de MySQL ejecutará al inicio cualquier script que encuentre en `/docker-entrypoint-initdb.d/

③ Montar una carpeta `html` de nuestro host en la ruta en la que el servicio `php` almacena la aplicación

`index.php` Descargar index.php

```
<script
src="https://gist.github.com/ualmtorres/0c9ba76eb22a35e328dbc322e6c100d1.js"></script>
```

`init.sql` Descargar init.sql

```
<script
src="https://gist.github.com/ualmtorres/eb328b653fcc5964f976b22c320dc10f.js"></script>
```

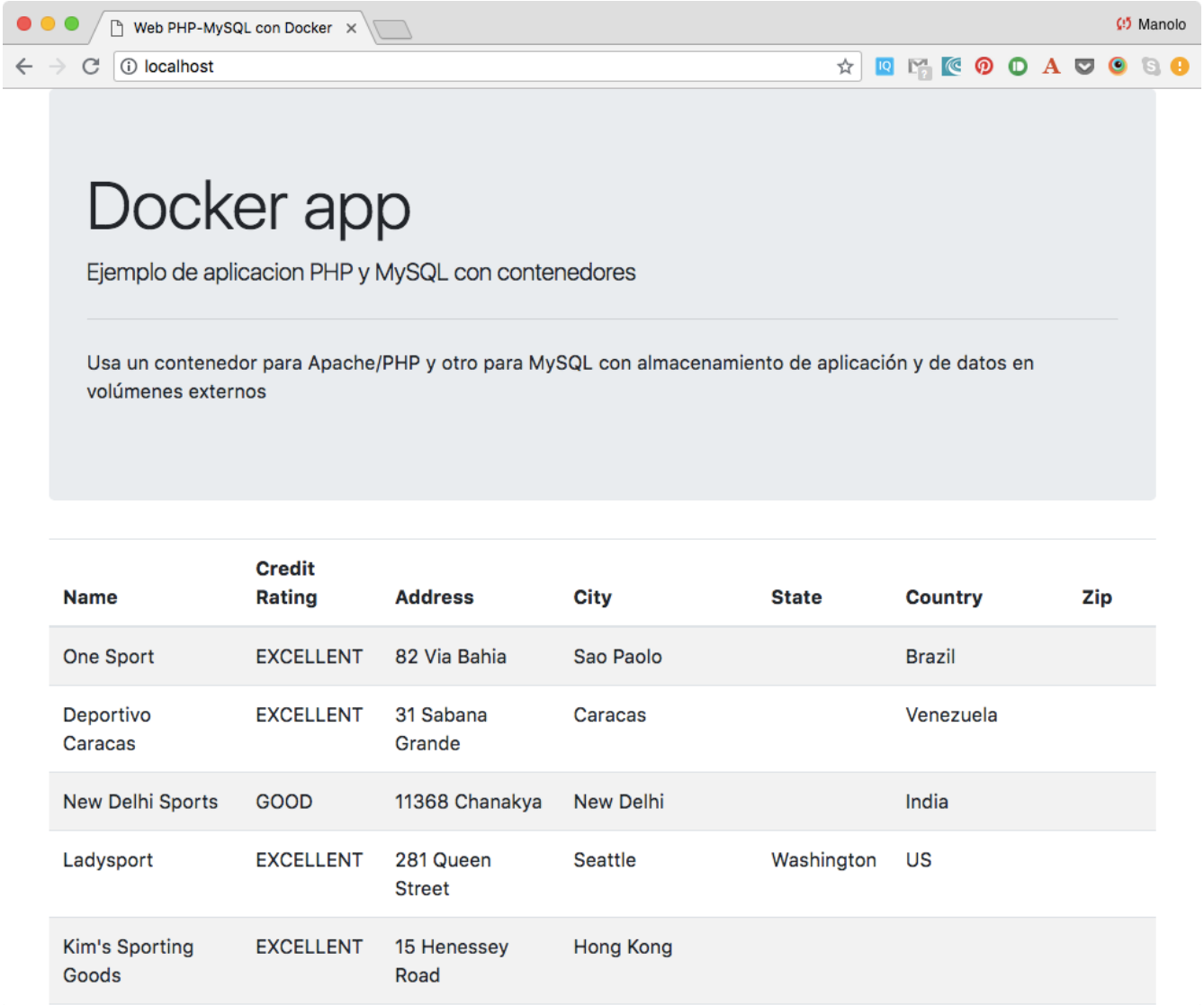


Figure 10. Aplicación web PHP que muestra listado de clientes almacenados en MySQL

Chapter 6. Plataformas de gestión de contenedores

Hasta ahora hemos estado usando Docker a través de Docker CLI. Con Docker CLI creamos contenedores, los mostramos, los detenemos, gestionamos volúmenes, redes, stacks, y demás operaciones.

Veamos dos Web UI para la gestión de contenedores como son Portainer y Rancher 1.6. Ambas se distribuyen mediante contedores.

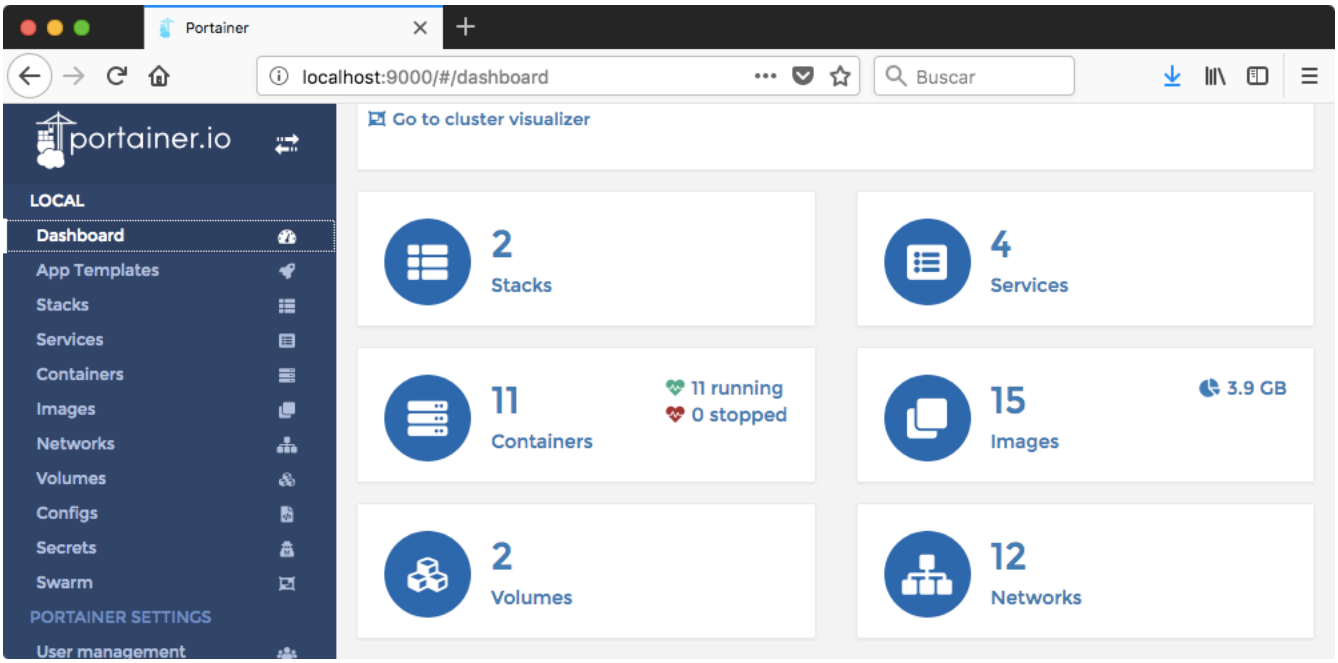
6.1. Administración de Docker con *Portainer*

Portainer es una Web UI sencilla y potente para administración de entornos Docker locales y remotos. Permite la administración de stacks, servicios, contenedores, imágenes, redes y volúmenes.

Para ejecutar *Portainer* con una carpeta local para el almacenamiento de los datos de Portainer (p. ej. usuarios) ejecutaríamos el comando siguiente:

```
docker run -d \
-p 9000:9000 \
-v "/var/run/docker.sock:/var/run/docker.sock" \
-v `pwd`/portainer_data:/data portainer/portainer \
portainer/portainer
```

Tras unos instantes tendremos Portainer en el puerto 9000. Después de definir una cuenta de usuario podremos entrar a administrar nuestro entorno Docker local.



6.2. Administración de Docker con Rancher 1.6 (Rancher Server)

Rancher dispone actualmente de dos versiones con funcionalidades diferentes: 1.6 y 2.0

Rancher 1.6 nos permite gestionar las imágenes, contenedores, stacks, volúmenes y demás objetos de Docker.

Rancher 2.0 va más allá y además permite gestionar clusters, en especial clusters de Kubernetes en múltiples hosts.

Para ejecutar Rancher 1.6 con una carpeta local para el almacenamiento de los datos de Rancher ejecutaríamos el comando siguiente

```
docker run -d --restart=unless-stopped \
-p 8080:8080 \
-v `pwd`/rancher16_data:/var/lib/mysql \
rancher/server ①
```

① `rancher/server` Es el nombre de la imagen de Rancher 1.6, mientras que `rancher/rancher` es el nombre de la imagen de Rancher 2.0

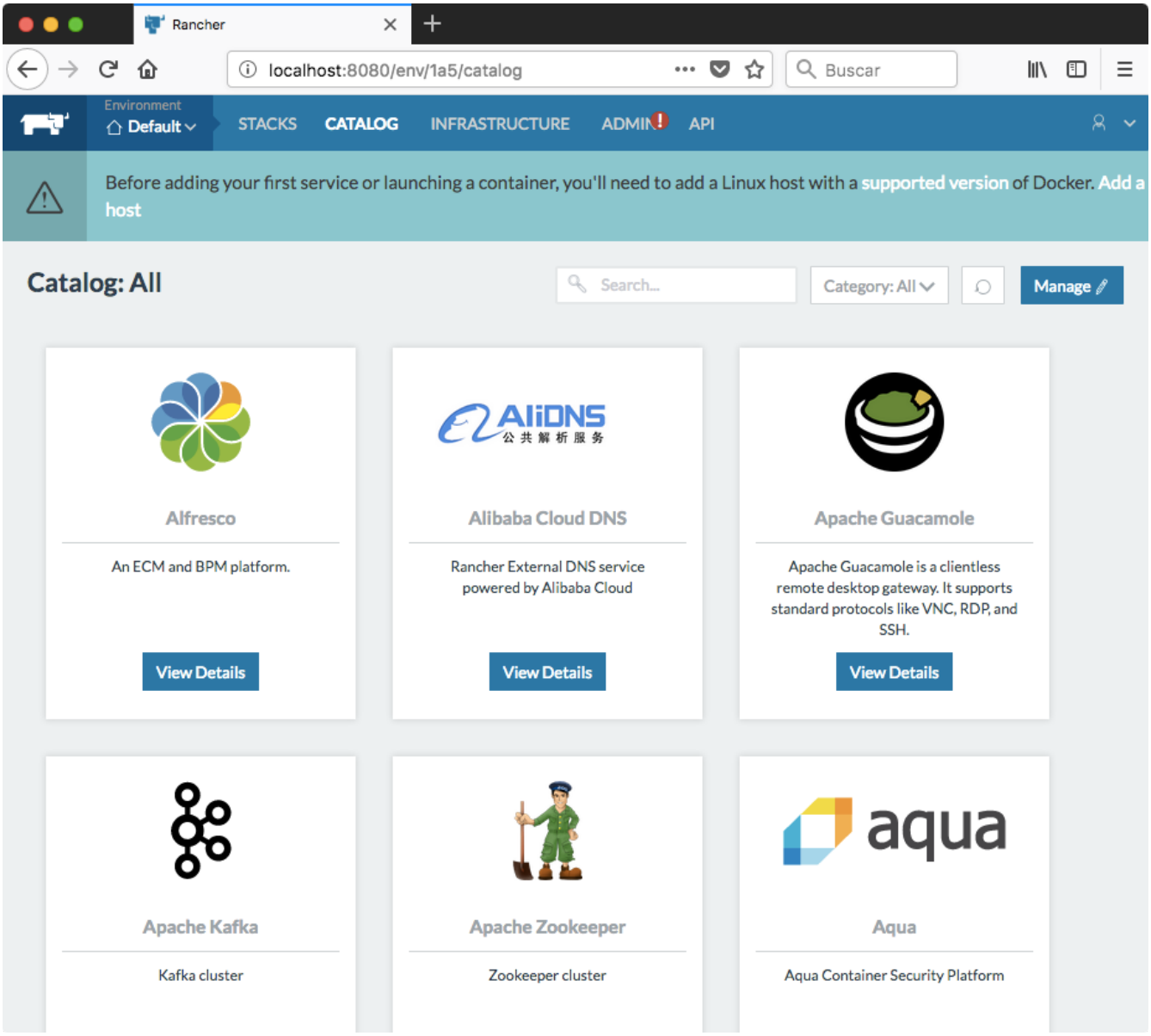


Figure 11. Catálogo de imágenes de Rancher 1.6

Chapter 7. Escalado de aplicaciones con Docker Swarm

- En aplicaciones distribuidas y con gran demanda podemos replicar contenedores en un servicio.
- Llegado el caso, necesitamos indicar la cantidad de contenedores que están ejecutando un servicio.
- También, ajustaremos la cantidad de los recursos del host que se dedican a la ejecución de las réplicas.
- Un balanceador de carga se encargará de ir alternando los contenedores a los que se envían las peticiones.

7.1. Orquestación de contenedores

Herramientas que nos ayudan en las tareas de:

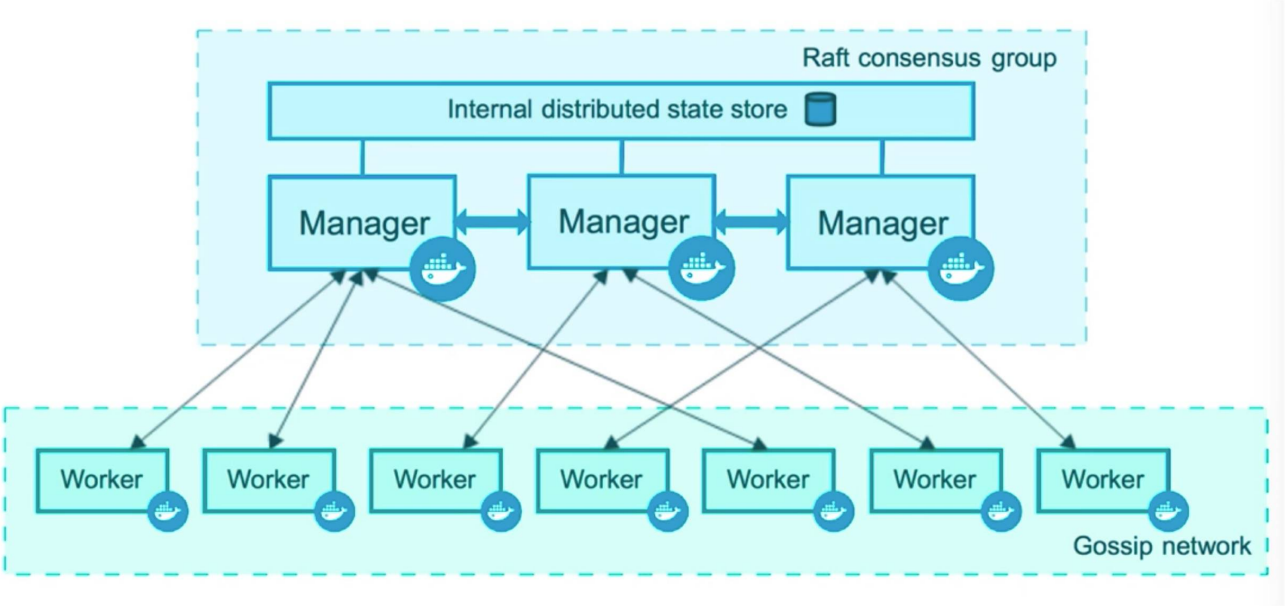
- Aprovisionamiento de hosts
- Instanciación de contenedores
- Sustitución de contenedores erróneos
- *Service discovery*
- Escalado aumentando o disminuyendo el número de contenedores
- Configuración de red
- Balanceo de carga

7.2. Orquestadores de contenedores más populares

- Amazon EC2 Container Service
- Azure Container Service
- Docker Swarm (el que veremos en este seminario debido a su sencillez)
- Kubernetes (el líder del momento)
- Google Container Engine (construido sobre Kubernetes)

7.3. Docker Swarm

- Docker Swarm propone que algunos de los conceptos de contenedores en un solo host sean válidos para convertirlo en un cluster (p. ej. redes overlay VXLAN)
- Un cluster de contenedores se ejecuta en un *swarm* (enjambre).
- Un *swarm* es una colección de Docker engines.



- Docker Swarm permite crear y gestionar clusters de contenedores usando el archivo `docker-compose.yml`.
- Un *swarm* está formado por nodos, que pueden ser máquinas físicas o virtuales.
- Hay dos tipos de nodos: *manager* y *worker*.
  - Los nodos *manager* se encargan de mantener el estado del cluster y de planificar los servicios.
  - Los nodos *worker* solo se encargan de ejecutar contenedores. De forma predeterminada, al definir un *manager* también es *worker*.

NOTE

Podemos hacer que los *managers* no sean *workers* haciendo que su disponibilidad sea `drain`

```
$ docker node update --availability drain
```

Swarm llevará a otros nodos los trabajos en ejecución y no le asignará nuevos trabajos.

- La composición del *swarm* es dinámica. Se pueden añadir y eliminar nodos *worker* sobre la marcha según sea conveniente. También es posible añadir nuevos nodos *manager*.

NOTE

Los nodos *worker* pueden ser promovidos a *manager* con `docker node promote` y los *manager* pueden ser degradados a *worker* con `docker node demote`.

7.3.1. Creación del swarm

Comenzamos creando una carpeta nueva que podemos denominar `swarm` para almacenar los archivos del proyecto (`docker-compose.yml`, código de la aplicación, ...)

A continuación ejecutamos el comando

`docker swarm init`

Esto crea un *swarm* de un nodo configurando como *manager* la máquina sobre la que se ha ejecutado. Además, muestra las instrucciones para añadir nuevos nodos *worker* o *manager* al *swarm* creado.

```
$ docker swarm init
Swarm initialized: current node (uifjsdvl3v1ydv5p7ocif2j13) is now a manager.

To add a worker to this swarm, run the following command:

    docker swarm join --token SWMTKN-1-
6635wxwy4wun1fvedd3hq27cganpqh28g0zh72ufhrytduewe9-1f6wj5wlzmjyt87ykdoyb1nci
192.168.65.3:2377
```

NOTE

Si olvidamos este token lo podemos volver a obtener con

```
$ docker swarm join-token worker
To add a worker to this swarm, run the following command:

    SWMTKN-1-6635wxwy4wun1fvedd3hq27cganpqh28g0zh72ufhrytduewe9-
1f6wj5wlzmjyt87ykdoyb1nci 192.168.65.3:2377
```

7.3.2. Definición de los servicios y réplicas

En el archivo `docker-compose.yml` definiremos cada uno de los servicios de nuestra aplicación, número de réplicas de los servicios y límites de recursos (CPU, RAM) asignados a cada contenedor.

Ejemplo de `docker-compose.yml`

```
version: '3'
services:
  php:
    image: ualmtorres/phalcon-apache-ubuntu
    ports:
      - "80:80"
    volumes:
      - ./html:/var/www/html
    deploy:
      replicas: 3
      resources:
        limits:
          cpus: "0.1"
          memory: 50M
      restart_policy:
        condition: on-failure
    networks:
```

```
      - webnet
  visualizer:
    image: dockersamples/visualizer:stable
    ports:
      - "8080:8080"
    volumes:
      - "/var/run/docker.sock:/var/run/docker.sock"
    networks:
      - webnet
networks:
  webnet:
```

En este caso definimos dos servicios: `php` y `visualizer`.

- `php` Tendrá 3 réplicas. A cada una de ellas le limitamos los recursos al 10% de uso de la CPU del host en el que se ejecuta el contenedor (también se pueden indicar qué núcleos usar) y 50 MB de RAM.
- `visualizer` Nos permite crear un contenedor que de forma sencilla muestra la cantidad y el estado de los contenedores de cada nodo del *swarm*.

A modo de ejemplo nuestra aplicación mostrará simplemente el ID del contenedor donde se está ejecutando para poder ver funcionando el balanceador.

`html/index.php`

```
<?php
 echo "Contenedor: " . gethostname();
?>
```

7.3.3. Despliegue del entorno (*stack*)

Para lanzar esta aplicación ejecutaremos el comando siguiente:

`docker stack deploy -c docker-compose.yml my_app`

El parámetro `-c` es opcional y especifica el archivo *compose*. `my_app` es el nombre que le damos al *stack* creado. Pensemos en un `stack` como un conjunto de servicios.

Tras unos instantes se creará el entorno y estarán ejecutándose la aplicación (puerto 80) y el visualizador (puerto 8080).

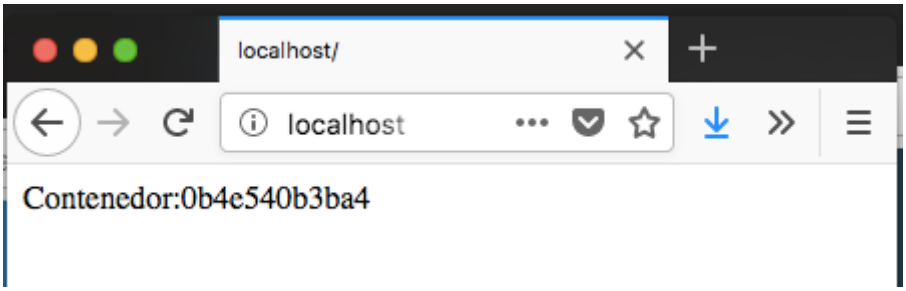


Figure 12. Aplicación mostrando el número de contenedor

Figure 13. El visualizador

Tenemos varios comandos para conocer el estado del `stack` creado.

Con `docker stack` podemos gestionar *stacks*. Por ejemplo, con `docker stack ls` vemos los stacks creados con la cantidad de servicios que incluye cada uno.

```
$ docker stack ls
```

```
NAME                SERVICES
my_app              2
```

Con `docker service ls` vemos los distintos servicios y la cantidad y estado de sus réplicas.

```
$ docker service ls
ID                  NAME                MODE                REPLICAS            IMAGE
PORTS
l6gwxu4asxb9        my_app_php          replicated          3/3
ualmtorres/phalcon-apache-ubuntu:latest           *:80->80/tcp
27l66joutbnd        my_app_visualizer   replicated          1/1
dockersamples/visualizer:stable   *:8080->8080/tcp
```

Con `docker stack ps my_app` vemos el estado de cada una de las tareas (contenedores) del *stack*.

```
$ docker stack ps my_app
ID                  NAME                  IMAGE                              NODE
DESIRED STATE       CURRENT STATE            ERROR               PORTS
0kc4fcw7bmva        my_app_visualizer.1   dockersamples/visualizer:stable   linuxkit-
025000000001   Running             Running 6 minutes ago
ueb5qs8kb0u6        my_app_php.1          ualmtorres/phalcon-apache-ubuntu:latest
linuxkit-025000000001   Running             Running 6 minutes ago
uejgm1a4035i        my_app_php.2          ualmtorres/phalcon-apache-ubuntu:latest
linuxkit-025000000001   Running             Running 6 minutes ago
nb0cbp5jhail        my_app_php.3          ualmtorres/phalcon-apache-ubuntu:latest
linuxkit-025000000001   Running             Running 6 minutes ago
```

7.4. Recuperación automática ante errores (*Self-healing*)

Veamos como al realizar una operación `kill` sobre uno de los contenedores, tras unos instantes vuelve a crearse un nuevo contenedor en su puesto, garantizando el número de réplicas especificado.

Primero mostramos los contenedores actuales

```
$ docker ps
CONTAINER ID        IMAGE                              COMMAND             CREATED
STATUS              PORTS               NAMES
ef0248683123        dockersamples/visualizer:stable   "npm start"         12 minutes
ago      Up 12 minutes       8080/tcp
my_app_visualizer.1.0kc4fcw7bmvale0i9unh2ph2m
4be7ebee6e84        ualmtorres/phalcon-apache-ubuntu:latest           "/run.sh"
12 minutes ago      Up 12 minutes       80/tcp
my_app_php.1.ueb5qs8kb0u6538vwtjc3piym
0b4e540b3ba4        ualmtorres/phalcon-apache-ubuntu:latest           "/run.sh"
```

```
12 minutes ago      Up 12 minutes       80/tcp
my_app_php.2.uejgm1a4035iz0bx208mxom9q
06a6011f4407        ualmtorres/phalcon-apache-ubuntu:latest           "/run.sh"
12 minutes ago      Up 12 minutes       80/tcp
my_app_php.3.nb0cbp5jhailiu3ee2bbvkbr7
```

Ahora lanzamos un `kill` sobre el tercer contenedor `06a6011f4407`

```
$ docker kill 06a6011f4407
```

Tras unos instantes habrá un nuevo contenedor en su puesto

```
$ docker ps
CONTAINER ID        IMAGE                              COMMAND             CREATED
STATUS              PORTS               NAMES
847cf3ae0a1c        ualmtorres/phalcon-apache-ubuntu:latest           "/run.sh"
5 seconds ago       Up 4 seconds        80/tcp
my_app_php.3.2khgab4796zhprb17ku0uj68l
ef0248683123        dockersamples/visualizer:stable   "npm start"         14 minutes
ago      Up 14 minutes       8080/tcp
my_app_visualizer.1.0kc4fcw7bmvale0i9unh2ph2m
4be7ebee6e84        ualmtorres/phalcon-apache-ubuntu:latest           "/run.sh"
14 minutes ago      Up 14 minutes       80/tcp
my_app_php.1.ueb5qs8kb0u6538vwtjc3piym
0b4e540b3ba4        ualmtorres/phalcon-apache-ubuntu:latest           "/run.sh"
14 minutes ago      Up 14 minutes       80/tcp
my_app_php.2.uejgm1a4035iz0bx208mxom9q
```

7.5. Escalado de la aplicación

En Docker Swarm podemos aumentar o disminuir el número de réplicas de un servicio mediante comandos o volviendo a desplegar el stack modificando el número de réplicas.

7.5.1. Escalado mediante comandos

La sintaxis es

`$ docker service scale <SERVICE-ID>=<NUMBER-OF-TASKS>`

Por ejemplo, para que el número de réplicas del servicio `php` del stack `my_app` sea 7 ejecutaríamos el comando

```
$ docker service scale my_app_php=7
my_app_php scaled to 7
overall progress: 7 out of 7 tasks
1/7: running   [==================================================>]
2/7: running   [==================================================>]
```

```
3/7: running   [==================================================>]
4/7: running   [==================================================>]
5/7: running   [==================================================>]
6/7: running   [==================================================>]
7/7: running   [==================================================>]
verify: Service converged


$ docker ps
CONTAINER ID        IMAGE                               COMMAND             CREATED
STATUS              PORTS               NAMES
81d00cc913c9        ualmtorres/phalcon-apache-ubuntu:latest           "/run.sh"
42 seconds ago      Up 43 seconds       80/tcp
my_app_php.7.tjknquh9jkj8bey90pisul2fw
0f9d8ba7b254        ualmtorres/phalcon-apache-ubuntu:latest           "/run.sh"
42 seconds ago      Up 43 seconds       80/tcp
my_app_php.5.daf2ni5cefc22zsky8ez7z58w
226c60af9984        ualmtorres/phalcon-apache-ubuntu:latest           "/run.sh"
42 seconds ago      Up 45 seconds       80/tcp
my_app_php.4.br2nbqhhh9s3x0fwo4d9llgco
e98b5194787b        ualmtorres/phalcon-apache-ubuntu:latest           "/run.sh"
42 seconds ago      Up 42 seconds       80/tcp
my_app_php.6.je2gqb380r3f6hcb6w9sd081q
847cf3ae0a1c        ualmtorres/phalcon-apache-ubuntu:latest           "/run.sh"
11 minutes ago      Up 11 minutes       80/tcp
my_app_php.3.2khgab4796zhprb17ku0uj68l
ef0248683123        dockersamples/visualizer:stable   "npm start"         25 minutes
ago      Up 25 minutes       8080/tcp
my_app_visualizer.1.0kc4fcw7bmvale0i9unh2ph2m
4be7ebee6e84        ualmtorres/phalcon-apache-ubuntu:latest           "/run.sh"
25 minutes ago      Up 25 minutes       80/tcp
my_app_php.1.ueb5qs8kb0u6538vwtjc3piym
0b4e540b3ba4        ualmtorres/phalcon-apache-ubuntu:latest           "/run.sh"
25 minutes ago      Up 25 minutes       80/tcp
my_app_php.2.uejgm1a4035iz0bx208mxom9q
```

7.5.2. Escalado volviendo a desplegar el stack

Para escalar con la técnica de *re-despliegue*, editar el archivo `docker-compose.yml` con el nuevo número de réplicas y volver a hacer el despliegue con

`docker stack deploy -c docker-compose.yml my_app`

Por ejemplo, probemos a reducir a 5 el número de réplicas. Con `docker stack ps my_app` podemos ver los cambios, así como con `docker ps`, así como desde Visualizer.

```
$ docker ps
CONTAINER ID        IMAGE                               COMMAND             CREATED
STATUS              PORTS               NAMES
0f9d8ba7b254        ualmtorres/phalcon-apache-ubuntu:latest           "/run.sh"
```

```
2 minutes ago       Up 2 minutes        80/tcp
my_app_php.5.daf2ni5cefc22zsky8ez7z58w
226c60af9984        ualmtorres/phalcon-apache-ubuntu:latest           "/run.sh"
2 minutes ago       Up 2 minutes        80/tcp
my_app_php.4.br2nbqhhh9s3x0fwo4d9llgco
847cf3ae0a1c        ualmtorres/phalcon-apache-ubuntu:latest           "/run.sh"
13 minutes ago      Up 13 minutes       80/tcp
my_app_php.3.2khgab4796zhprb17ku0uj68l
ef0248683123        dockersamples/visualizer:stable   "npm start"         27 minutes
ago      Up 27 minutes       8080/tcp
my_app_visualizer.1.0kc4fcw7bmvale0i9unh2ph2m
4be7ebee6e84        ualmtorres/phalcon-apache-ubuntu:latest           "/run.sh"
27 minutes ago      Up 27 minutes       80/tcp
my_app_php.1.ueb5qs8kb0u6538vwtjc3piym
0b4e540b3ba4        ualmtorres/phalcon-apache-ubuntu:latest           "/run.sh"
27 minutes ago      Up 27 minutes       80/tcp
my_app_php.2.uejgm1a4035iz0bx208mxom9q
```

TIP Esta operación de actualización del despliegue es la que también se usa para añadir nuevos servicios a un *stack*. Basta con añadir los nuevos servicios a `docker-compose.yml` y *redesplegar* el stack

7.5.3. Apagado de la aplicación y del swarm

Para eliminar el stack de dos servicios creado para este ejemplo ejecutamos el comando siguiente

```
$ docker stack rm my_app
```

Esta operación detendrá todos los contendores asociados al stack.

Para que nuestro nodo (el nodo *manager*) deje el *swarm* ejecutaremos el comando

```
$ docker swarm leave --force
```

7.6. Resumen de los comandos para *swarms*

```
$ docker swarm init             Inicialización de swarm y del nodo manager
$ docker stack deploy -c docker-compose.yml <stack> Despliegue de stack
$ docker stack ls               Lista de stacks y cantidad de servicios que tiene
$ docker service ls             Listado de servicios y estado de sus réplicas
$ docker stack ps <stack>       Listado de las tareas del stack
$ docker stack rm <stack>       Eliminación del stack
$ docker swarm leave --force    Salida de un nodo del swarm
```

Chapter 8. Docker machine

Docker machine es una herramienta que nos permite:

- Administrar *swarms* aprovisionándolos y añadiéndoles nodos
- Instalar y ejecutar Docker en los nodos creados
- Aprovisionar los nodos creados

Los nodos del *swarm* pueden ser máquinas virtuales creadas en el host con VirtualBox o con proveedores cloud como Azure, AWS u OpenStack. En este seminario nos centraremos en la creación e inicialización de un *swarm* en OpenStack. En nuestro caso, Docker machine usará la API de OpenStack encargándose de la creación de los nodos del *swarm* evitando tener que crear los nodos desde OpenStack.

8.1. Creación de archivo con variables de entorno

Incluiremos las opciones habituales y que están disponibles como variables de entorno. Se incluirán en el archivo los valores de OpenStack relativos a la cuenta de usuario, red, proyecto y demás.

`openrc-mtorres.sh`

```
export OS_USERNAME=mtorres
export OS_PASSWORD=XXXXXXXXXXXX
export OS_PROJECT_NAME=mtorres
export OS_USER_DOMAIN_NAME=Default
export OS_PROJECT_DOMAIN_NAME=Default
export OS_AUTH_URL=http://www.xxx.yyy.zzz:5000/v3
export OS_IDENTITY_API_VERSION=3
export OS_IMAGE_API_VERSION=2
export OS_TENANT_NAME=mtorres
```

donde `www.xxx.yyy.zzz` es el nombre DNS o IP que usemos para conectarnos a OpenStack.

A continuación, cargaremos las variables de entorno con

`source openrc-mtorres.sh`

8.2. Creación de los nodos del *swarm*

Para crear los nodos del *swarm* en OpenStack con Docker machine tendremos que pasar una serie de valores relativos al sabor, nombre de imagen, red, nombre de usuario de las instancias, claves ssh, y demás.

Desde una máquina que esté en la red de la UAL creamos una máquina con Docker con este comando (no vale por VPN porque actualmente el puerto 5000 que se usa para la autenticación con OpenStack no está abierto en VPN):

```
docker-machine create -d openstack \
--openstack-flavor-name small \
--openstack-image-name "Ubuntu 16.04 LTS" \
--openstack-domain-name default \
--openstack-net-name mtorres-net \
--openstack-floatingip-pool ext-net \
--openstack-ssh-user ubuntu \
--openstack-sec-groups default \
--openstack-keypair-name mtorres_ual \
--openstack-private-key-file ~/.ssh/id_rsa \
nodo1
```

Esto comenzará a crear una instancia con los parámetros indicados en nuestro proyecto OpenStack. Tras unos instantes, nos devolverá esta información relativa a la creación del `nodo1`.

```
Running pre-create checks...
Creating machine...
(nodo1) Creating machine...
Waiting for machine to be running, this may take a few minutes...
Detecting operating system of created instance...
Waiting for SSH to be available...
Detecting the provisioner...
Provisioning with ubuntu(systemd)...
Installing Docker...
Copying certs to the local machine directory...
Copying certs to the remote machine...
Setting Docker configuration on the remote daemon...
Checking connection to Docker...
Docker is up and running!
To see how to connect your Docker Client to the Docker Engine running on this virtual
machine, run: docker-machine env nodo1
```

Repetimos el comando para crear otro nodo en nuestro proyecto al que denominaremos `nodo2`.

Para listar las dos máquinas creadas con Docker machine ejecutaremos el comando siguiente.

```
$ docker-machine ls
NAME    ACTIVE   DRIVER      STATE     URL                         SWARM   DOCKER
ERRORS
nodo1   -        openstack   Running   tcp://192.168.66.211:2376           v18.05.0-ce
nodo2   -        openstack   Running   tcp://192.168.66.235:2376           v18.05.0-ce
```

Para acceder a las máquinas creadas y ver que aún no tienen contenedores creados debemos cargar las variables de entorno de la que vayamos a usar (p. ej. `nodo1`).

```
$ eval $(docker-machine env nodo1)
$ docker ps
```

```
CONTAINER ID        IMAGE               COMMAND             CREATED             STATUS
PORTS               NAMES
```

Para crear el *swarm* haremos que el nodo 1 sea el *manager* y el nodo 2 sea el *worker*.

8.2.1. Creación del nodo *manager*

```
$ docker-machine  ssh nodo1 "sudo docker swarm init --advertise-addr 192.168.66.211"
Swarm initialized: current node (y0831vf8yj3vu120jj3zp8c6k) is now a manager.

To add a worker to this swarm, run the following command:

    docker swarm join --token SWMTKN-1-
1j411qkevgcza9uunune32q4p6p4xylyz944uozow0l7shr66t-7ujr8f9i9lti19rz9oqkjh89n
192.168.66.211:2377

To add a manager to this swarm, run 'docker swarm join-token manager' and follow the
instructions.
```

Anotaremos el token para poder añadir nodos al swarm.

8.2.2. Creación del nodo *worker*

```
$ docker-machine ssh nodo2 "sudo docker swarm join --token SWMTKN-1-
1j411qkevgcza9uunune32q4p6p4xylyz944uozow0l7shr66t-7ujr8f9i9lti19rz9oqkjh89n
192.168.66.211:2377"
This node joined a swarm as a worker.
```

8.3. Comprobación del *swarm* creado

```
$ docker-machine ssh nodo1 "sudo docker node ls"
ID                            HOSTNAME            STATUS              AVAILABILITY
MANAGER STATUS      ENGINE VERSION
y0831vf8yj3vu120jj3zp8c6k *   nodo1               Ready               Active
Leader              18.05.0-ce
zgc0e50822qabzlfceclrso6c     nodo2               Ready               Active
18.05.0-ce
```

8.4. Despliegue de la aplicación en los nodos

1. Abrir sesiones SSH en cada nodo del *swarm* para añadir el usuario `ubuntu` al grupo `docker` con `sudo usermod -a -G docker ubuntu` y añadir los directorios que usen como punto de montaje de los volúmenes (Probar a hacer esto con docker-machine)
2. Hacer despliegue desde el nodo manager

```
version: '3'
services:
  php:
    image: ualmtorres/phalcon-apache-ubuntu
    ports:
      - "80:80"
    volumes:
      - ./html:/var/www/html
    deploy:
      replicas: 4
      resources:
        limits:
          cpus: "0.1"
          memory: 50M
      restart_policy:
        condition: on-failure
    networks:
      - webnet
  visualizer:
    image: dockersamples/visualizer:stable
    ports:
      - "8080:8080"
    volumes:
      - "/var/run/docker.sock:/var/run/docker.sock"
    deploy:
      placement:
        constraints: [node.role == manager]
    networks:
      - webnet
networks:
  webnet:
```

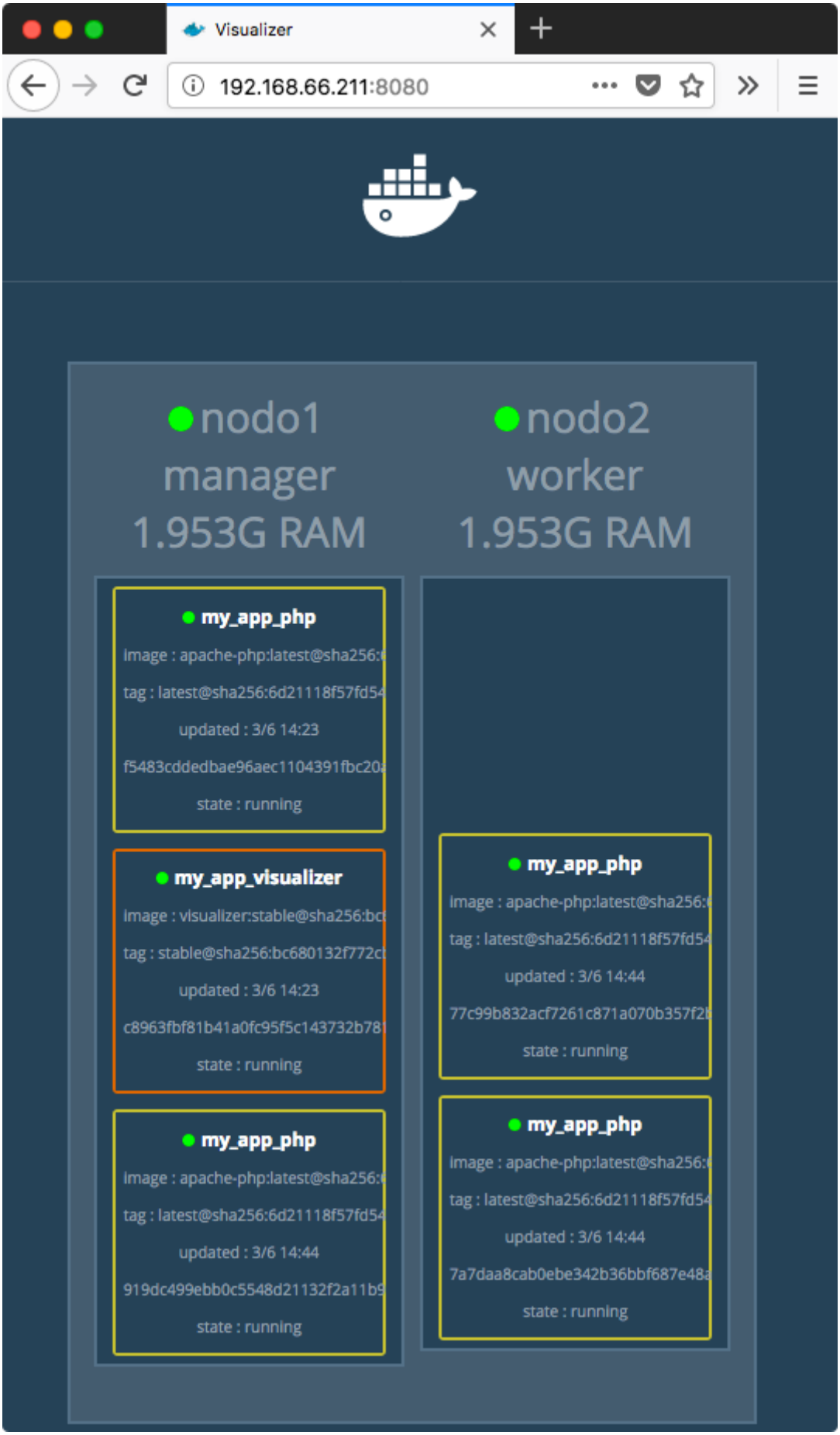


Figure 14. Cluster de dos nodos con 4 réplicas del servicio `php`

8.5. Desalojo de un nodo

Para finalizar vamos a ver cómo desalojar un nodo (p. ej. debido a una operación de mantenimiento en uno de los servidores del *swarm*).

Por ejemplo, veamos como desalojar el nodo *worker* `nodo2`.

Haremos esta operación directamente desde el nodo *manager*, aunque se podría hacer desde otro nodo, o incluso de forma remota con `docker-machine`. Tras iniciar sesión en `nodo1` primero veremos el estado de los servicios y los nodos en los que se están ejecutando.

```
$ docker service ps my_app_php
ID                  NAME                IMAGE                     NODE
DESIRED STATE       CURRENT STATE         ERROR                               PORTS
phqzhyvrn2cs        my_app_php.1        ualmtorres/phalcon-apache-ubuntu:latest
nodo1               Running             Running 5 days ago
7tcammel94a8        my_app_php.2        ualmtorres/phalcon-apache-ubuntu:latest
nodo2               Running             Running 5 days ago
5y969fj92o5g        my_app_php.3        ualmtorres/phalcon-apache-ubuntu:latest
nodo1               Running             Running 5 days ago
m7cjujgg08bw        my_app_php.4        ualmtorres/phalcon-apache-ubuntu:latest
nodo2               Running             Running 5 days ago
```

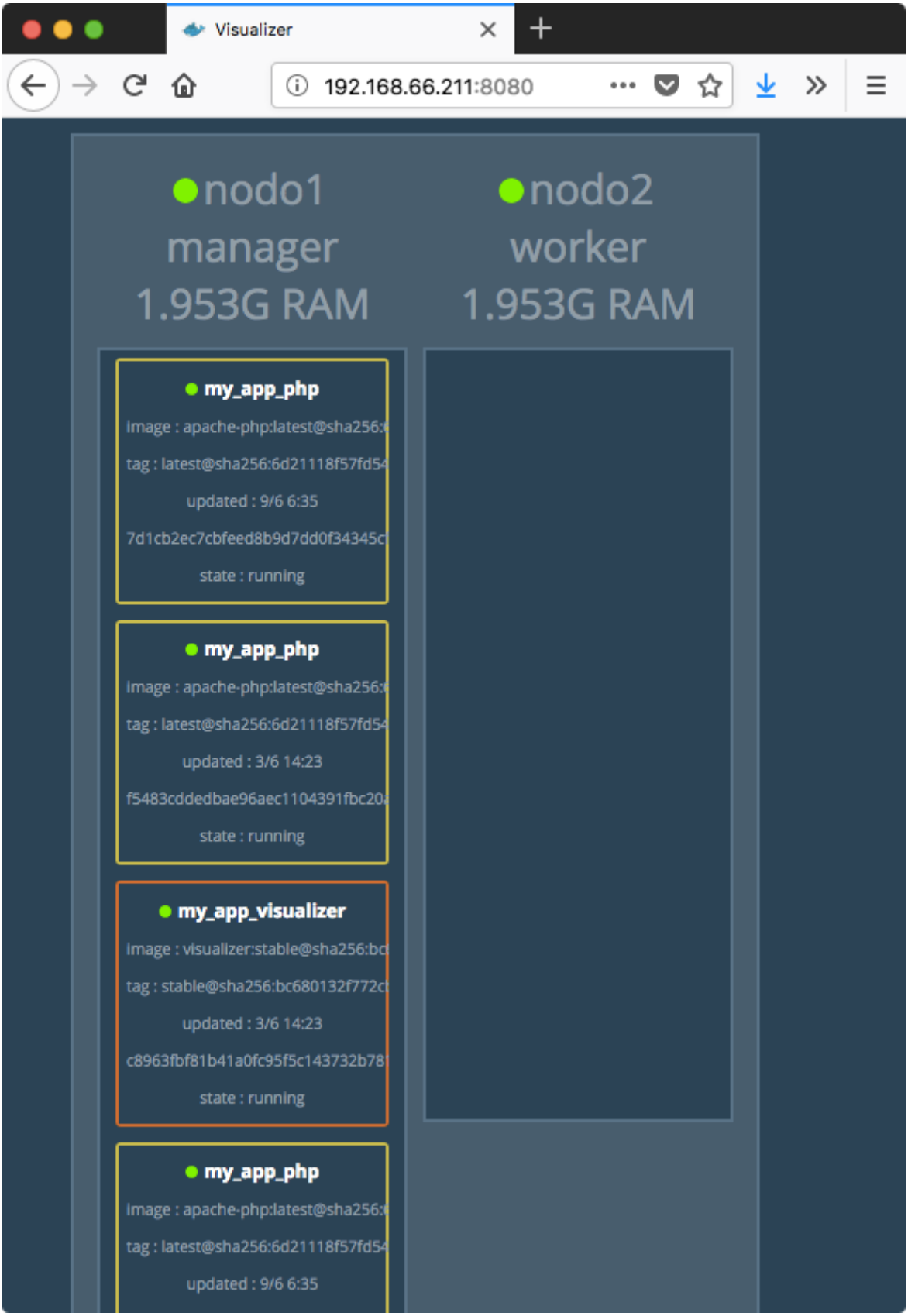
Haremos la operación `drain` para desalojar el `nodo2`

```
$ docker node update --availability drain nodo2
```

Al comprobar el estado del servicio veremos que ahora todos los contenedores han pasado a `nodo1` manteniendo el número de réplicas que tuviésemos.

```
$ docker service ps my_app_php
ID                  NAME                IMAGE                     NODE
DESIRED STATE       CURRENT STATE            ERROR                               PORTS
phqzhyvrn2cs        my_app_php.1        ualmtorres/phalcon-apache-ubuntu:latest
nodo1               Running             Running 5 days ago
zqb89jdjraz7        my_app_php.2        ualmtorres/phalcon-apache-ubuntu:latest
nodo1               Running             Starting 2 seconds ago
5y969fj92o5g        my_app_php.3        ualmtorres/phalcon-apache-ubuntu:latest
nodo1               Running             Running 5 days ago
6q9955ce8fdw        my_app_php.4        ualmtorres/phalcon-apache-ubuntu:latest
nodo1               Running             Starting 3 seconds ago
```

Con Visualizer podemos ver de forma más gráfica cómo se ha desalojado el nodo



Una vez finalizada la operación de mantenimiento volveríamos a poner el nodo como activo (`--availability active`)

```
$ docker node update --availability active nodo2
```

El nodo ahora podrá volver a recibir nuevas tareas

Chapter 9. Creación de clusters Kubernetes con Rancher 2.0

Kubernetes se ha convertido en el estándar para orquestación de contenedores. La mayoría de los proveedores cloud lo ofrece con infraestructura estándar.

Rancher 2.0 nos ofrece una plataforma on-premise para hacer despliegues de clusters de Kubernetes en proveedores cloud como Amazon EKS, Google Container Engine, Azure Container Service, Amazon EC2, Microsoft Azure, Digital Ocean, OpenStack, RackSpace y demás,

NOTE RancherOS es una distribución de Linux rápida y ligera optimizada para usarse con contenedores. Incluye el software mínimo para ejecutar Docker.

Para ejecutar Rancher 2.0 con una carpeta local para el almacenamiento de los datos de Rancher ejecutaríamos el comando siguiente

```
docker run -d --restart=unless-stopped \
-p 9000:9000 \
-v `pwd`/rancher16_data:/var/lib/mysql \
rancher/rancher ①
```

① `rancher/rancher` Es el nombre de la imagen de Rancher 2.0, mientras que `rancher/server` es el nombre de la imagen de Rancher 1.6

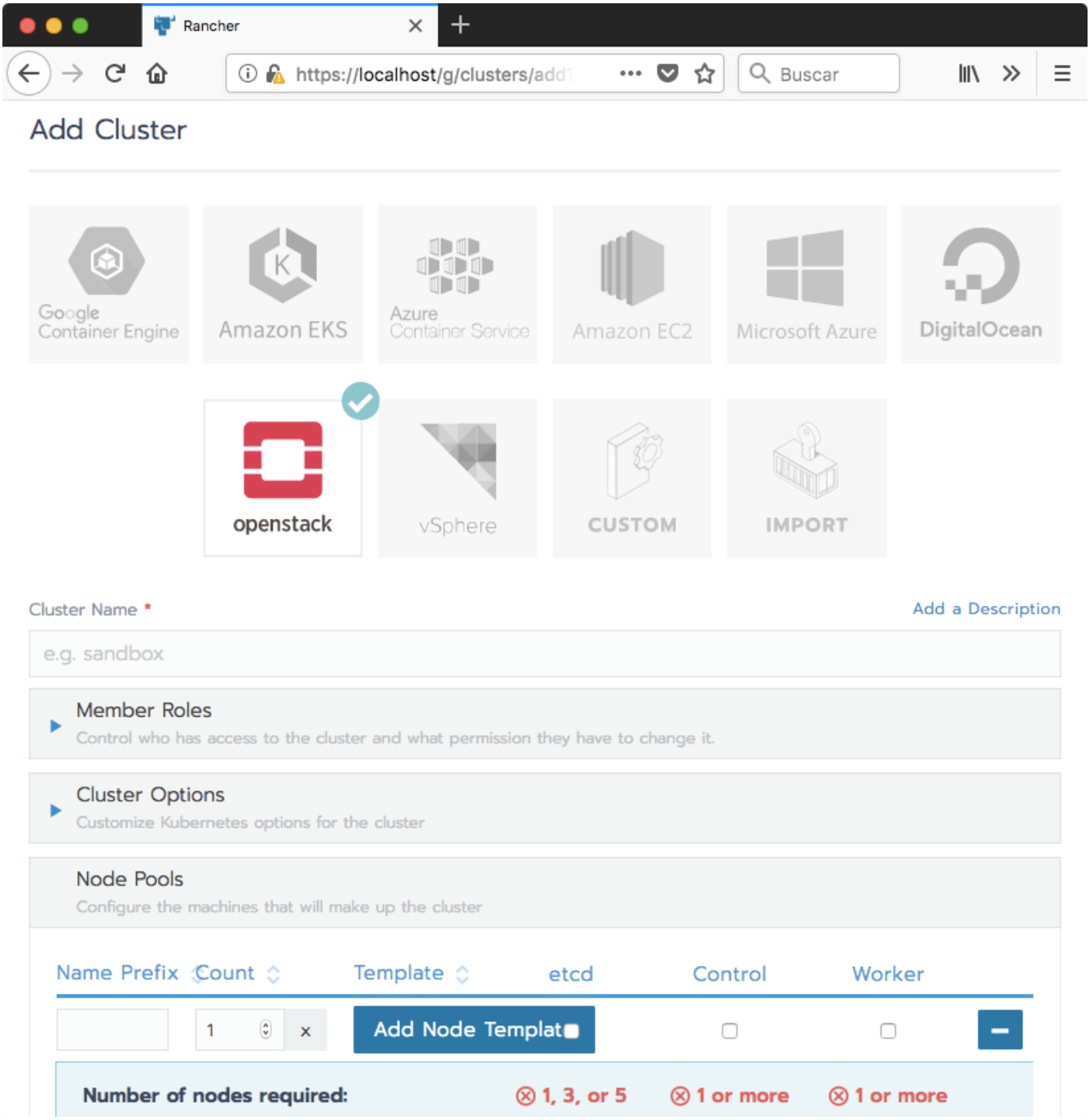


Figure 15. Elección de OpenStack como infraestructura para despliegue de un cluster de contenedores

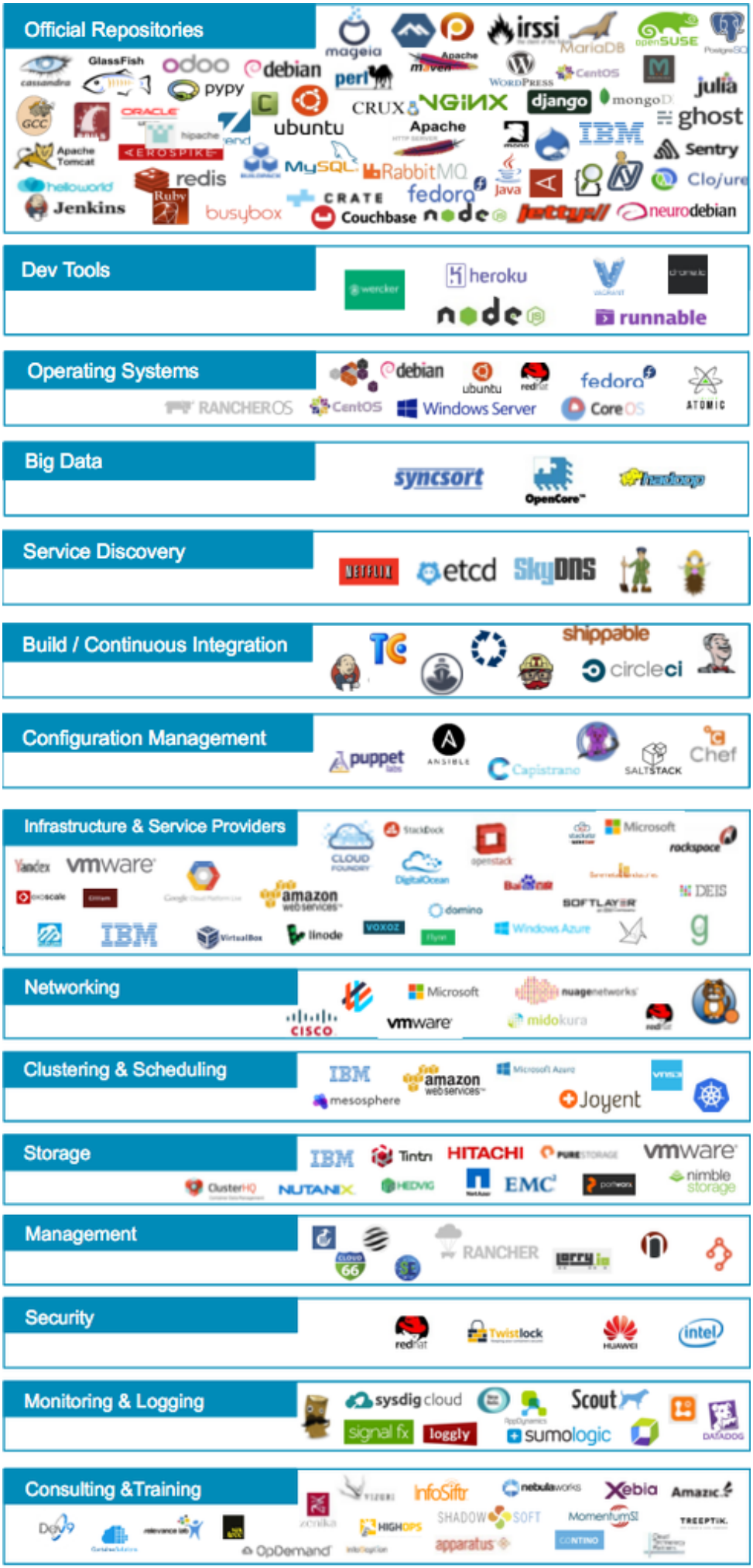
Chapter 10. Otras cosas interesantes

10.1. Microservicios y contenedores

Con micro servicios:

- Establecemos un contrato, normalmente mediante una API REST, versionada para no romper funcionalidad a usuarios anteriores
- Ocupan un tamaño reducido y suelen realizar una tarea muy concreta
  - Autenticación,
  - API REST. Toda la API vs. cada endpoint
  - Estadísticas consumo de recursos
  - Exportar salida a central de logs
  - …
- Dockerizar con cabeza
  - Comenzamos pasando todo nuestro sistema o MV a un contenedor Docker. Con solo eso ya conseguimos ejecutar nuestro sistema en distintas máquinas con distintos SO y configuraciones.
  - No intentar pasar de una vez de aplicación monolítica a micro servicios diminutos

10.2. El ecosistema Docker


Official Repositories
Dev Tools
Operating Systems
Big Data
Service Discovery
Build / Continuous Integration
Configuration Management
Infrastructure & Service Providers
Networking
Clustering & Scheduling
Storage
Management
Security
Monitoring & Logging
Consulting &Training

10.3. Instalar un registro de imágenes propio

Es posible tener un registro propio para imágenes por cuestiones de seguridad y confidencialidad. Veamos cómo crear un registro propio mediante contenedores (uno para el registro en sí y otro cono Web UI).

El ejemplo será obtener una imagen Alpine de Docker Hub y subirla a nuestro propio registro

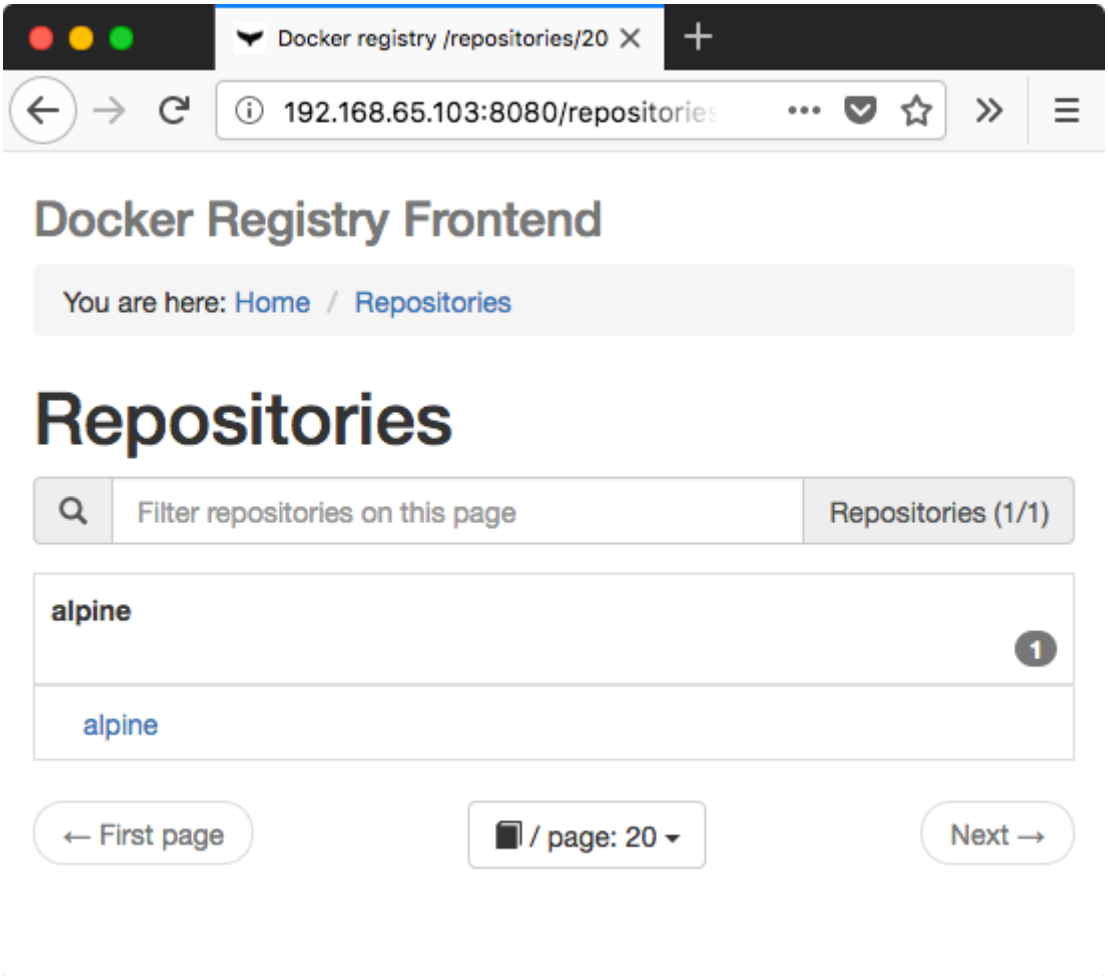
```
// En el servidor (p.e. 192.168.65.103)
$ docker run -d -p 5000:5000 --restart always --name registry registry:2
$ docker run \
  -d \
  -e ENV_DOCKER_REGISTRY_HOST=192.168.65.103 \
  -e ENV_DOCKER_REGISTRY_PORT=5000 \
  -p 8080:80 \
  konradkleine/docker-registry-frontend:v2

// En nuestro equipo
$ docker pull alpine ①
$ docker image list | grep alpine ②
$ docker tag 3e640a41799a 192.168.65.103:5000/alpine ③
$ docker push 192.168.65.103:5000/alpine ④
```

① Descargar una imagen de prueba de Alpine al registro local

② Obtener el identificador de la imagen Alpine descargada (p.e. `3e640a41799a`)

③ La imagen se etiqueta añadiéndole como prefijo host:puerto de nuestro registro

④ Subida de la imagen al registro



10.4. Conexión a daemons Docker remotos

Util para conectarnos desde nuestro equipo al Docker de producción o al de pruebas

- Crear en local un archivo de variables de entorno (p.e. `DockerProyectoBrainstorm.sh`)

```
export DOCKER_TLS_VERIFY=1
export DOCKER_CERT_PATH="<ruta completa de certificado>"
export DOCKER_HOST="tcp://<IP o nombre DNS>:443"
```

- Después, `source DockerProyectoBrainstorm.sh` y ¡¡Estamos conectados!!
- Si la conexión fuera abierta, indicaríamos `export DOCKER_TLS_VERIFY=0`.

10.5. Uso de HAProxy

```
docker-compose.yml
version: '3'
services:
  php:
    image: ualmtorres/phalcon-apache-ubuntu
    ports:
      - "80"
```

```
    environment:
     - SERVICE_PORTS=80 ①
    volumes:
      - ./html:/var/www/html
    deploy:
      replicas: 6
      resources:
        limits:
          cpus: "0.1"
          memory: 50M
      restart_policy:
        condition: on-failure
    networks:
      - webnet
  visualizer:
    image: dockersamples/visualizer:stable
    ports:
      - "8080:8080"
    volumes:
      - "/var/run/docker.sock:/var/run/docker.sock"
    deploy:
      placement:
        constraints: [node.role == manager]
    networks:
      - webnet
  proxy: ②
    image: dockercloud/haproxy
    depends_on:
      - php ③
    environment:
      - BALANCE=leastconn ④
    volumes:
      - /var/run/docker.sock:/var/run/docker.sock
    ports:
      - 80:80
    networks:
      - webnet
    deploy:
      placement:
        constraints: [node.role == manager]
networks:
  webnet:
```

① Cambio en la configuración de puertos para usar HAProxy

② Servicio `proxy`

③ No crear el servicio `proxy` hasta que no se haya creado el `php`

④ Usa la política de balanceo *least connections* en lugar de *round robin* la cual es la predeterminada